

Prompt 4 — Planning Pantalla Completa + Asignación Masiva de Habitaciones en Grupos

PARTE 1 — Botón de pantalla completa para el planning

El objetivo

El botón actual del sidebar (SidebarTrigger) oculta el menú lateral. Se pide un segundo botón que además oculte el header, dando máximo espacio vertical al planning.

1a — Agregar estado `planningFullscreen` en el layout principal

En el componente raíz del layout (donde está `SidebarProvider` , cerca de la línea 558), agregar:

```
const [planningFullscreen, setPlanningFullscreen] = useState(false);
```

1b — Modificar el header para que se oculte en fullscreen

```
{/* Reemplazar el header existente por: */}
<header className={`flex items-center justify-between gap-4 px-4 py-2 border-b bg
  planningFullscreen ? "h-0 overflow-hidden py-0 border-0" : ""
}`}>
  <SidebarTrigger data-testid="button-sidebar-toggle" />
  <div className="flex items-center gap-3">
    {/* ... contenido existente del header sin cambios ... */}
  </div>
</header>
```

1c — Pasar el estado al router/planning via contexto o URL param

La forma más simple: usar un `Context` o simplemente detectar la ruta. Como solo el planning necesita el botón, lo más limpio es agregar el botón dentro de la página del planning y usar CSS.

Opción más simple — CSS fullscreen nativo del browser:

En `planning.tsx`, agregar un botón de pantalla completa que use la Fullscreen API del navegador:

```
const [isFullscreen, setIsFullscreen] = useState(false);
const planningRef = useRef<HTMLDivElement>(null);

const toggleFullscreen = async () => {
  if (!document.fullscreenElement) {
    await planningRef.current?.requestFullscreen();
    setIsFullscreen(true);
  } else {
    await document.exitFullscreen();
    setIsFullscreen(false);
  }
};

useEffect(() => {
  const handleFullscreenChange = () => {
    setIsFullscreen(!document.fullscreenElement);
  };
  document.addEventListener("fullscreenchange", handleFullscreenChange);
  return () => document.removeEventListener("fullscreenchange", handleFullscreenC
}, []);
```

1d — Agregar ref y botón en `planning.tsx`

Envolver el contenido de la página en un `div` con el ref:

```
return (
  <div ref={planningRef} className={`flex flex-col gap-4 p-6 h-full ${isFullscree
    <div className="flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-be
      <div>
        <h1 className="text-3xl font-bold tracking-tight" data-testid="text-plann
          Planning de Ocupación
        </h1>
        <p className="text-muted-foreground">Vista de disponibilidad por habitaci
      </div>
      <div className="flex items-center gap-2 flex-wrap">
        /* ... botones existentes de navegación ... */

        /* NUEVO: Botón pantalla completa */
        <Button
          variant="outline"
          size="icon"
          onClick={toggleFullscreen}
```

```

        title={isFullscreen ? "Salir de pantalla completa" : "Pantalla completa"}
        data-testid="button-fullscreen-planning"
      >
        {isFullscreen ? (
          <Minimize2 className="h-4 w-4" />
        ) : (
          <Maximize2 className="h-4 w-4" />
        )}
      </Button>
    </div>
  </div>
  { /* ... resto del contenido sin cambios ... */ }
</div>
);

```

Agregar `Maximize2`, `Minimize2` y `useRef` a los imports.

PARTE 2 — Asignación masiva de habitaciones en grupos

El problema actual

`AssignRoomDialog` asigna una sola habitación por vez. Si un bloque tiene 5 habitaciones, hay que hacer clic en "Asignar" 5 veces.

El objetivo

Al hacer clic en "Asignar" en un bloque, abrir un dialog que muestre N filas (una por habitación pendiente del bloque) donde en cada fila se elige la habitación y se escribe el nombre del ocupante.

2a — Reemplazar `AssignRoomDialog` por `AssignBlockDialog`

Crear un nuevo componente `AssignBlockDialog` que reemplaza al actual. Puede estar en el mismo archivo `group-detail.tsx`.

```

function AssignBlockDialog({
  group,
  block,
  open,
  onOpenChange,
  onSuccess,
}: {
  group: GroupWithDetails;

```

```

    block: GroupRoomBlockWithDetails;
    open: boolean;
    onOpenChange: (open: boolean) => void;
    onSuccess: () => void;
  }) {
    const { toast } = useToast();

    // Calcular cuántas habitaciones faltan asignar
    const assignedCount = group.reservations.filter(r =>
      // Reservas que pertenecen a este bloque (mismo tipo de hab y fechas del grup
      r.room?.roomId === block.roomId
    ).length;
    const pending = Math.max(0, block.quantity - assignedCount);

    const defaultCheckIn = block.blockCheckInDate || group.checkInDate;
    const defaultCheckOut = block.blockCheckOutDate || group.checkOutDate;

    // Filas de asignación: una por habitación pendiente
    const [rows, setRows] = useState<Array<{
      roomId: string;
      firstName: string;
      lastName: string;
    }>>()
    Array.from({ length: pending }, () => ({ roomId: "", firstName: "Sin Asignar"
    });

    const { data: rooms } = useQuery<RoomWithType[]>({
      queryKey: ["/api/rooms"],
    });

    const availableRooms = rooms?.filter(
      (r) => r.roomId === block.roomId && r.status === "available"
    ) || [];

    // Habitaciones ya elegidas en otras filas (para no duplicar)
    const chosenRoomIds = rows.map(r => r.roomId).filter(Boolean);

    const [isSubmitting, setIsSubmitting] = useState(false);

    const handleAssignAll = async () => {
      const validRows = rows.filter(r => r.roomId);
      if (validRows.length === 0) {
        toast({ title: "Seleccioná al menos una habitación", variant: "destructive"
        return;
      }
    }

    setIsSubmitting(true);
  }

```

```

let successCount = 0;
let failCount = 0;

for (const row of validRows) {
  try {
    await apiRequest("POST", `/api/groups/${group.id}/assign-room`, {
      roomId: row.roomId,
      guestFirstName: row.firstName || "Sin Asignar",
      guestLastName: row.lastName || "",
      ratePlanId: block.ratePlanId,
    });
    successCount++;
  } catch {
    failCount++;
  }
}

setIsSubmitting(false);
queryClient.invalidateQueries({ queryKey: ["/api/groups", group.id] });
queryClient.invalidateQueries({ queryKey: ["/api/rooms"] });

if (failCount === 0) {
  toast({ title: `${successCount} habitación(es) asignada(s) exitosamente` })
  onSuccess();
  onOpenChange(false);
} else {
  toast({
    title: `${successCount} asignadas, ${failCount} fallaron`,
    variant: "destructive",
  });
}
};

return (
  <Dialog open={open} onOpenChange={onOpenChange}>
    <DialogContent className="max-w-2xl max-h-[85vh] overflow-y-auto">
      <DialogHeader>
        <DialogTitle>
          Asignar Habitaciones - {block.roomType.name}
        </DialogTitle>
        <DialogDescription>
          Bloque de {block.quantity} habitaciones. {assignedCount} ya asignadas
          {defaultCheckIn && `Check-in: ${defaultCheckIn} | Check-out: ${defau
        </DialogDescription>
      </DialogHeader>

      <div className="space-y-3 py-2">

```

```

{rows.length === 0 ? (
  <div className="text-center py-8 text-muted-foreground">
    <CheckCircle2 className="h-8 w-8 mx-auto mb-2 text-green-500" />
    <p>Todas las habitaciones del bloque ya están asignadas.</p>
  </div>
) : (
  <>
    <div className="grid grid-cols-[2fr_1fr_1fr_auto] gap-2 text-xs font-sans">
      <span>Habitación</span>
      <span>Nombre</span>
      <span>Apellido</span>
      <span></span>
    </div>
    {rows.map((row, index) => (
      <div key={index} className="grid grid-cols-[2fr_1fr_1fr_auto] gap-2">
        <Select
          value={row.roomId}
          onChange={(value) => {
            const updated = [...rows];
            updated[index].roomId = value;
            setRows(updated);
          }}
        >
          <SelectTrigger data-testid={`select-room-${index}`}>
            <SelectValue placeholder="Seleccionar hab." />
          </SelectTrigger>
          <SelectContent>
            {availableRooms
              .filter(r => !chosenRoomIds.includes(r.id) || r.id === row.roomId)
              .map((room) => (
                <SelectItem key={room.id} value={room.id}>
                  Hab. {room.roomNumber} – Piso {room.floor}
                </SelectItem>
              ))
            }
            {availableRooms.filter(r => !chosenRoomIds.includes(r.id) |
              <SelectItem value="" disabled>Sin disponibilidad</SelectItem>
            )}
          </SelectContent>
        </Select>

        <Input
          placeholder="Nombre"
          value={row.firstName}
          onChange={(e) => {
            const updated = [...rows];
            updated[index].firstName = e.target.value;

```

```

        setRows(updated);
    }}
    data-testid={`input-firstname-${index}`}
  />

  <Input
    placeholder="Apellido"
    value={row.lastName}
    onChange={(e) => {
      const updated = [...rows];
      updated[index].lastName = e.target.value;
      setRows(updated);
    }}
    data-testid={`input-lastname-${index}`}
  />

  <Button
    variant="ghost"
    size="icon"
    className="h-8 w-8 text-muted-foreground hover:text-destructi
    onClick={() => setRows(rows.filter((_, i) => i !== index))}
    title="Quitar fila"
    data-testid={`button-remove-row-${index}`}
  >
    <X className="h-4 w-4" />
  </Button>
</div>
)))

{/* Botón para agregar más filas si se quiere asignar más */}
{rows.length < availableRooms.length && (
  <Button
    variant="ghost"
    size="sm"
    className="text-xs"
    onClick={() => setRows([...rows, { roomId: "", firstName: "Sin
    data-testid="button-add-assignment-row"
  >
    <Plus className="h-3 w-3 mr-1" />
    Agregar otra habitación
  </Button>
  )}
</>
)}
</div>

<DialogFooter>

```

```

        <Button variant="outline" onClick={() => onOpenChange(false)}>Cancelar<
        {rows.length > 0 && (
          <Button
            onClick={handleAssignAll}
            disabled={isSubmitting || rows.every(r => !r.roomId)}
            data-testid="button-confirm-assign-all"
          >
            {isSubmitting ? (
              <><Loader2 className="h-4 w-4 mr-2 animate-spin" />Asignando...</
            ) : (
              `Asignar ${rows.filter(r => r.roomId).length} habitación(es)`
            )}
          </Button>
        )}
      </DialogFooter>
    </DialogContent>
  </Dialog>
);
}

```

2b — Reemplazar uso de `AssignRoomDialog` por `AssignBlockDialog`

En `GroupDetailPage`, reemplazar:

```

// De:
{assigningBlock && (
  <AssignRoomDialog
    group={group}
    block={assigningBlock}
    open={!assigningBlock}
    onOpenChange={(open) => !open && setAssigningBlock(null)}
    onSuccess={() => {}}
  />
)}

// Por:
{assigningBlock && (
  <AssignBlockDialog
    group={group}
    block={assigningBlock}
    open={!assigningBlock}
    onOpenChange={(open) => !open && setAssigningBlock(null)}
    onSuccess={() => queryClient.invalidateQueries({ queryKey: ["/api/groups", gr
  />
)}

```


El componente `AssignRoomDialog` viejo puede quedar en el archivo (no hace falta eliminarlo) o eliminarse si se desea limpiar.

2c — Agregar imports necesarios en `group-detail.tsx`

```
import { X, Plus, CheckCircle2, Loader2 } from "lucide-react";
```

Resumen de archivos modificados

Archivo	Cambio
<code>client/src/pages/planning.tsx</code>	<code>useRef</code> , <code>toggleFullscreen</code> , botón <code>Maximize2/Minimize2</code>
<code>client/src/pages/group-detail.tsx</code>	Nuevo componente <code>AssignBlockDialog</code> , reemplazar uso de <code>AssignRoomDialog</code>